Step 1

Create a new folder and open it in VSCode

Create a results.py file

Enter this code

```
def calculate_result(marks):
    marks.sort()
    for mark in marks:
        if mark < 0:
            raise ValueError("Invalid mark")
    average = sum(marks) // len(marks)

    if average > 50:
        status = "Pass"
    else:
        status = "Fail"

    return {
        "average": average,
        "status": status
    }
```

The function is intended to return a dictionary containing an average and a pass/fail status. Do not fix anything yet.

Step 2

Create test_results.py. Paste the following code and save it:

```
import pytest
from results import calculate_result
def test_passing_result():
    result = calculate_result([60, 70, 80])

    assert result == {
        "average": 70,
        "status": "Pass"
    }

def test_failing_result():
    result = calculate_result([20, 30, 40])

    assert result == {
        "average": 30,
        "status": "Fail"
    }

def test_negative_mark_is_rejected():
    with pytest.raises(ValueError):
        calculate_result([-10, 60])
```

run the baseline tests

Open Terminal → New Terminal.

Type:

python -m pytest -v

expect:

test_passing_result PASSED

test_failing_result PASSED

test_negative_mark_is_rejected PASSED

Step 3 – Ask Copilot for a review

Open chat and attach the 2 files as context.

You need to create a prompt which asks copilt to review the 2 files.

Think about:

How correct the calculations are, how accurate the pass / fail decision is

Input validation, missing or incorrect tests

What do you need to be told about each finding or issue?

Use these requirements as a guide

1. Accept a non-empty list of marks.
2. Each mark must be an integer between 0 and 100 inclusive.
   Boolean values such as True and False are invalid.
3. Invalid input must raise a ValueError.
4. Calculate the average using all the marks.
5. An average of 50 or above means "Pass";
   below 50 means "Fail".
6. Return the average rounded to two decimal places.
   Determine pass/fail using the unrounded average.
7. Do not change the original list.
8. Keep the existing function name and parameter.

********SPOILER********

Appendix 1 has an example prompt

Review the response. It’s findings are claims to investigate

Step 4 – Verify the pass / fail boundary

Input this prompt:

> Investigate the pass/fail boundary for the input [50].
>
> State:
>
> 1. The expected status from the requirements.
>
> 2. The comparison used in the current code.
>
> 3. The status the current code produces.
>
> Do not modify any files.

Copilot should explain

```
Expected status: Pass.
Current comparison: average > 50.
For an average of 50, this condition is false.
Current status: Fail.
```

Input this command into the terminal

python -c "from results import calculate_result; print(calculate_result([50]))"

Expected response - {'average': 50, 'status': 'Fail'}

You have confirmed the finding by comparing observed behaviour with the requirement.

Step 5 – verify the average calculation

Use this prompt

```
Investigate the average calculation for [50, 51].

Explain the correct average, what // does here, and
the  average returned by the current code.

Identify the requirement violated.

Do not modify any files.
```

Investigate the response – something along the lines of

Correct average: (50 + 51) / 2 = 50.5.

The code uses //, which performs floor division.

Current average: 50.

The calculation does not meet the average requirement.

In a terminal run

python -c "from results import calculate_result; print(calculate_result([50, 51]))"

{'average': 50, 'status': 'Fail'}

The incorrect average is confirmed. The "Fail" status also reflects the existing boundary problem.

Step 8 – Investigate the Input-list mutation

As above, investigate the finding around input list mutation

******SPOILER******

Appendix 2 shows the prompts and expected outputs

Step 9 - Ask Copilot to add a test before fixing the code.

Paste this prompt

```
Add one pytest test to test_results.py.
The test must call calculate_result with [50] and assert that
the status is "Pass".

Keep all existing tests.
Do not modify results.py.
Do not add other tests or refactor anything.

Show the proposed change.
```

test_results.py should contain this additional test, or an equivalent one:

```
def test_average_of_50_is_a_pass():
       result = calculate_result([50])
       assert result["status"] == "Pass"
```

In a terminal, run

python -m pytest -v

The new test fails:

test_average_of_50_is_a_pass FAILED

The failure includes a comparison similar to: AssertionError: assert 'Fail' == 'Pass'

The summary reports:

1 failed, 3 passed

The new test demonstrates the confirmed defect. This failure is expected.

Step 10 – Ask Copilot to fix only the boundary

Create a prompt which outlines what must be fixed, ensuring safeguards to other code and smallest changes possible

******SPOILER******

Appendix 3 has a possible prompt to do this

Run the tests again and verify the fix

Step 11 – Ask Copilot to review the updated code

Prompt

```
Review the current results.py and test_results.py.

Confirm:

1. Whether the pass/fail boundary is now correct.

2. Whether the new test checks that boundary.

3. Whether the change stayed within the requested scope.

4. Which previously identified defects remain.

Do not modify any files.
Explain whether four passing tests are sufficient to
approve the implementation against all the requirements.
```

Expected response – along the lines of

Review question	Expected conclusion
Is the boundary fixed?	Yes, `>= 50` includes exactly 50
Is it tested?	Yes, the new test uses `[50]`
Was the scope respected?	Yes, if only the comparison and test were changed
Are other defects present?	Yes: floor division, list mutation and incomplete validation
Should the whole implementation be approved?	Request changes because requirements remain unmet

Step 12 – Draft the evidence record

Paste the following prompt only if these results match what you actually observed. Otherwise, replace them with your actual results.

```
Draft a concise Code Review Evidence Record.

Observed evidence:

    - Original tests: 3 passed.

    - [50] initially returned average 50 and status "Fail".

    - [50, 51] returned average 50 instead of 50.5.

    - [80, 20, 60] changed to [20, 60, 80] after the function
    call.

    - After adding the boundary test: 1 failed, 3 passed.

    - After changing > to >=: 4 passed.

    - [50] now returns status "Pass".

Use these columns:

Finding | Evidence | Decision | Action
Distinguish fixed issues from unresolved issues.

Do not invent test results.
End with an approval recommendation and a reason.
Do not modify any files.
```

Appendix 1 –

Review results.py and test_results.py against the requirements below.
Do not edit any files or implement fixes yet.

Focus on:

- Correctness of the average calculation.

- Correctness of the pass/fail decision.

- Input validation.

- Unintended changes to the original list.

- Missing or incorrect tests.

For each finding, provide:

1. The relevant code.

2. The requirement it violates.

3. A concrete example showing the impact.

4. A suggested way to verify the finding.

Separate defects from optional improvements and questions.
Avoid duplicate findings.
Do not invent additional requirements.

Requirements:

1. Accept a non-empty list of marks.

2. Each mark must be an integer between 0 and 100 inclusive.

   Boolean values such as True and False are invalid.

3. Invalid input must raise a ValueError.

4. Calculate the average using all the marks.

5. An average of 50 or above means "Pass";

   below 50 means "Fail".

6. Return the average rounded to two decimal places.

   Determine pass/fail using the unrounded average.

7. Do not change the original list.

8. Keep the existing function name and parameter.

Appendix 2

Use this prompt

```
Investigate marks.sort() using the input [80, 20, 60].

 Explain:

1. Whether sorting is needed to calculate an average.

2. Whether marks.sort() changes the original list.

3. Which requirement is violated.

 Do not modify any files.
```

Cursor should explain that:

- Sorting is unnecessary for calculating the average.
- marks.sort() changes the list in place.
- This violates the requirement to leave the original list unchanged.

Run the following command in terminal:

python -c "from results import calculate_result; marks=[80,20,60]; print('Before:', marks); calculate_result(marks); print('After:', marks)"

Before: [80, 20, 60]

After: [20, 60, 80]

You have confirmed a third finding.

Appendix 3 –

```
Fix only the pass/fail boundary in results.py.

An average of exactly 50 must produce "Pass".

Make the smallest possible change.
Do not change the average calculation, input validation,
sorting statement, function name or parameter.

Do not refactor unrelated code.
Show the proposed change.
```

Inspect and apply the proposed edit, then save the file.